

POPULAR TRADITIONAL AND NEURAL NETWORK-BASED METHODS FOR SOLVING INVERSE KINEMATICS OF COMPLEX MANIPULATORS: A COMPUTATIONAL STUDY

SUMIT PATIDAR

UTKARSH KUNWAR

JANUARY 14, 2020

Abstract

Inverse kinematics solutions for complex systems are difficult to find and closed-form analytical solutions may not always exist. Resorting to iterative numerical methods requires heavy computation which may not be feasible in real-time applications. Numerical solutions sometimes also suffer from singularities which can lead to a misbehaving system while other solutions might have too slow a convergence. Iterative algorithms are used widely and solutions using artificial neural networks have also been introduced in recent years. Therefore, this project investigates the performances of solutions to the inverse kinematics problem using different popular iterative methods and neural networks. The performance of different architectures and the effect of hyper-parameters were compared against the Jacobian based iterative methods. Among the Jacobian methods, it was observed that for trivial trajectories, the Jacobian inverse-based solutions converged faster and were far more accurate than the Jacobian transpose-based method. For the inverse kinematic solution using neural network, it was observed that the proposed network architecture and custom loss function performed better than the state-of-the-art networks with standard mean-squared error for the case of 7 degree of freedom robotic manipulator. Compared to the iterative methods, the solution proposed by the network fell short in accuracy but performed on par in terms of execution time.

Keywords— inverse kinematics, 7 degree of freedom, iterative methods, neural network, custom loss

Contents

List of Figures	2
List of Tables	2
List of Symbols	3
1 Introduction	4
1.1 Theoretical framework/literature study	4
1.2 Research questions, hypotheses	5
2 Methodology	5
2.1 Procedure	5
2.2 Data collection	5
2.3 System specifications	6

2.3.1 Hardware	6
2.3.2 Software	6
2.4 Network hyperparameters	7
3 Results and Analysis	7
3.1 Iterative Methods	7
3.2 Artificial Neural Networks	10
4 Discussion and Conclusions	11
5 Future work	11

List of Figures

1 Popular Industrial robots	4
2 One layer perceptron network for 3DOF manipulator.	4
3 Proposed dense network architecture for 7DOF	6
4 Proposed CNN architecture for 7DOF	6
7 Performance of Jacobian methods on Linear trajectory	8
10 Performance of Jacobian methods on Square trajectory	9
13 Performance of Jacobian methods on Circular trajectory	9
14 MSE loss for 1 layer perceptron	10
15 MSE loss for custom network architectures.	10
16 Custom MSE loss and network performance	10

List of Tables

1 Hardware specifications of local computer	6
2 Software specifications for training neural network	7
3 Hyperparameters for training neural network	7

List of Acronyms and Abbreviations

LBR "Leichtbauroboter" (German for lightweight robot)

iiwa Intelligent Industrial Work Assistant

DOF Degree of Freedom

SCARA Selective Compliance Assembly Robot Arm

ROS Robotic Operating System

ANN Artificial Neural Network

SVD Singular Value Decomposition

iter Iterations

MSE Mean Square Error

GPU Graphical Processing Unit

CNN Convolutional Neural Network

CUDA Compute Unified Device Architecture

RNN Recurrent Neural Network

LSTM Long Short-term Memory

ReLU Rectified Linear Unit

3D Three Dimensional

List of Symbols

ε_{Θ} Error in joint angles

ε_X Error in pose

$\hat{\Theta}$ Estimated joint angles

$\hat{X}$ Estimated pose

J Jacobian matrix

J^T Jacobian transpose

J^{-1} Jacobian inverse

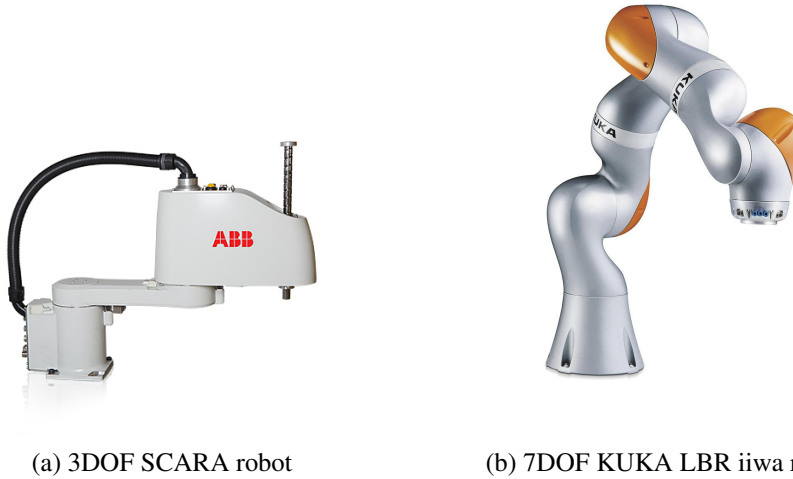
$J^{\dagger}$ Jacobian pseudo inverse

X Target pose

1 Introduction

Robotics is an interdisciplinary field of engineering which deals with the construction and control of robots and autonomous systems. It is one of the most popular and emerging engineering fields of the decade.

Robots typically have joints with certain degrees of freedom (DOF) and an end-effector to interact with the environment. The joint parameters are mapped to world coordinates of end-effector using a mathematical function and this process is known as forward-kinematics. In practical applications, the world positions to be reached by end-effector are known but the corresponding joint parameters are unknown. The process of finding those joint parameters from end-effector positions is known as inverse-kinematics.



(a) 3DOF SCARA robot

(b) 7DOF KUKA LBR iiwa robot

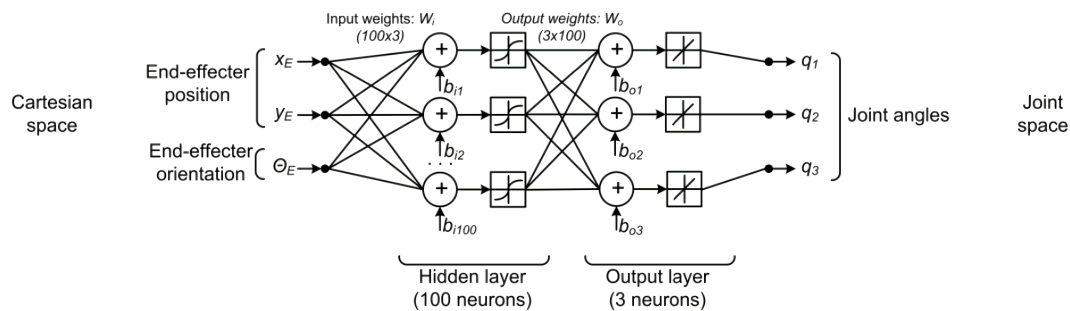
Source: ABB [1], KUKA [2]

Figure 1: Popular Industrial robots

1.1 Theoretical framework/literature study

This project builds on the idea of comparing well-performing algorithms for solving inverse kinematics of robotic systems described in [3]. We will be considering a model of KUKA LBR iiwa (7DOF) [2] manipulator for performing simulations in Gazebo [4] to test existing as well as proposed methods for this project. The Jacobian iterative methods are well formulated and widely used across the industry [5].

Recent advents in the field of machine learning and deep learning, thanks to the increase in computation power, has enabled research in several fields including robotics. Neural networks have been successfully used to obtain the inverse kinematic solutions of simpler robotic manipulator systems [6-8].



Source: A. Vasile, Duka [6]

Figure 2: One layer perceptron network for 3DOF manipulator.

1.2 Research questions, hypotheses

Artificial Neural Networks (ANNs) are as reactive and robust as iterative algorithms for solving inverse kinematics of complex manipulators.

2 Methodology

The study empirically used the iterative methods because for complex manipulators, an analytical solution is not feasible. Further, the study used the ANN architectures [6-8] and custom architectures to find if a faster and more accurate model could be created.

2.1 Procedure

1. At first, popular existing iterative (Jacobian based) [5] algorithms were implemented using Python and were integrated with Robot Operating System (ROS) [9] to carry out the simulations of the KUKA manipulator in Gazebo. In parallel, benchmarking of each method for execution speed and accuracy was done. The algorithm used for Jacobian based methods is as follows:

X : Target and $\hat{\Theta}$: Initial approximation

repeat

$$\begin{aligned}\hat{X} &= K(\hat{\Theta}), \\ \epsilon_X &= \hat{X} - X, \\ \epsilon_{\Theta} &= J^{-1}(\hat{\Theta})\epsilon_X, \\ \hat{\Theta} &= \hat{\Theta} - \epsilon_{\Theta},\end{aligned}\tag{1}$$

until $\epsilon_X \leq \text{tolerance}$

where, K denotes the forward kinematic function, J is the jacobian matrix, ϵ_X is the pose error (position and orientation error), and ϵ_{Θ} is the error in joint angles. Jacobian inverse is substituted by the right Moore-Penrose inverse of the Jacobian or by its transpose or using Singular Value Decomposition (SVD),

$$J^{\dagger} = J^T (JJ^T)^{-1}.\tag{2}$$

2. The next focus was to replicate the ANN architectures to obtain the inverse kinematics solutions and simulate the KUKA manipulator.
3. Formulation of new ANN architectures was carried out which was validated against state-of-the-art results. Figures 3 and 4 show the proposed network architectures that were used in this study.
4. Formulation of a custom loss function to better represent the minimisation problem at hand than the standard mean-squared error (MSE) provided by the deep learning libraries.

2.2 Data collection

For a given manipulator, in our case KUKA, data was generated using forward kinematics of the end-effector. The dataset contained 5 million mappings between pose and joint angles. Since forward kinematics is many to one mapping, the duplicate datapoints were removed and then the data was normalized between -1 to 1. This data was used to generate joint angles given the end-effector pose using the neural networks.

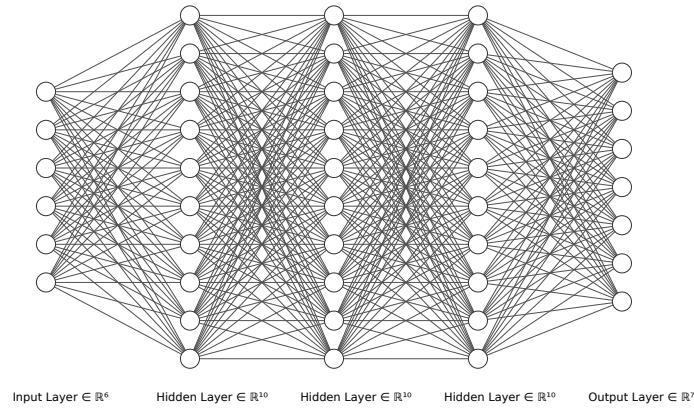


Figure 3: Proposed dense network architecture for 7DOF.

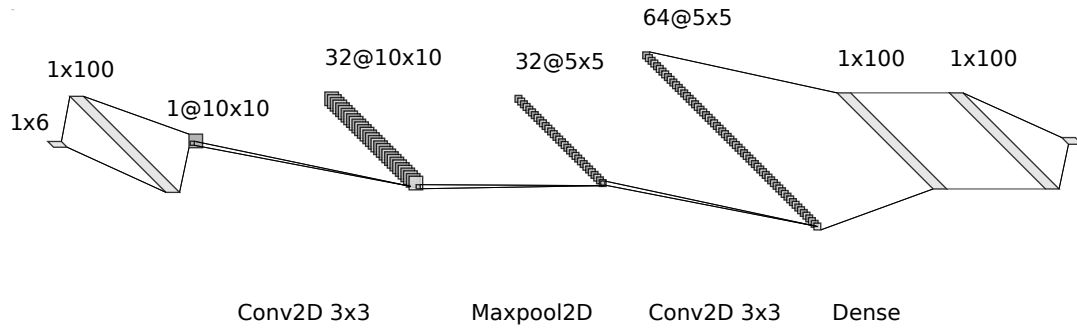


Figure 4: Proposed CNN architecture for 7DOF.

2.3 System specifications

2.3.1 Hardware

Table 1 shows a list of hardware specification on which the ROS and Gazebo simulations for the iterative part, and the training on the neural networks were performed.

Property	Specification		
	Local (ROS)	Local (ANN)	Cloud (ANN)
OS	Ubuntu 18.04.3 LTS	Ubuntu 18.04.3 LTS	Ubuntu 18.04.3 LTS
Kernel	4.9.210-generic x86_64	5.0.0-37-generic x86_64	4.14.137+ x86_64
Total cores	4	4	1
Model	Intel i7-6700	Intel Core i7-7700HQ (-MT-MCP-)	Intel Xeon (-MT-)
Clock speed	3400MHz	2800MHz	2300MHz
GPU	—	NVIDIA GM107GLM [Quadro M1200 Mobile 4GB]	NVIDIA GP100GL [Tesla P100 PCIe 16GB]

Table 1: Hardware specifications of local machine for ROS and Gazebo simulations.

For training on the local machine, the official Tensorflow image from Docker is used with the tag `latest-gpu-py3-jupyter`.

2.3.2 Software

Table 2 shows the versions of software used for the ROS and Gazebo simulations, and the versions of drivers and software used for training the neural networks both on Google Colab and local computer.

Software	Version		
	Local (ROS)	Local (ANN)	Cloud
ROS	melodic	—	—
Gazebo	9.0.0	—	—
Python [10]	2.7.17, 3.6.9	3.6.9	3.6.9
NumPy [11]	1.16.5	1.16.5	1.16.5
NVIDIA Driver [12]	—	440.33.01	418.67
CUDA [13]	—	10.2	10.1
Docker [14]	—	19.03.5	19.03.5
Tensorflow [15]	—	1.15.0	1.15.0
Tensorflow Graphics [16]	—	1.0.0	1.0.0

Table 2: Software specifications of Google Colab and local machine for training neural network.

2.4 Network hyperparameters

The hyperparameters for model training and compilation are mentioned in table 3.

Property	Value		
	Dense ₁	Dense ₂	CNN
Number of layers	3	2	5
Number of units	100	100	100 (in dense layers)
Activation function	ReLU-Sigmoid	ReLU-Sigmoid	ReLU-Sigmoid
Optimiser	Adam	Adam	Adam
Loss	MSE	Custom MSE	MSE
Type of minimisation	Output comparison	Direct minimisation	Output comparison

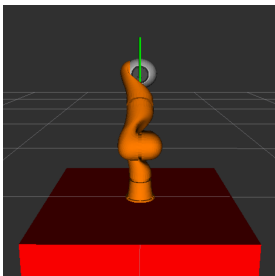
Table 3: Hyperparameters for training neural network.

3 Results and Analysis

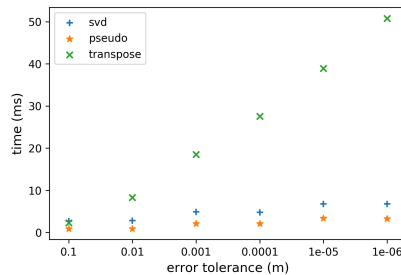
3.1 Iterative Methods

The error tolerance is defined as the magnitude of the error within which the solution provided by the method is considered acceptable and further iterations are stopped. The solution is accepted whenever either of these conditions are met. This helps ensure sufficient reactivity of the robotic manipulator system.

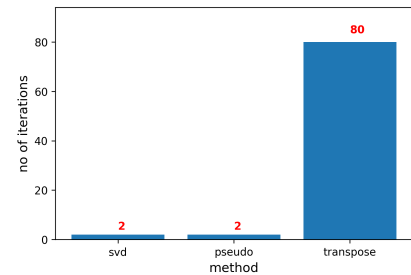
Figures 7, 10, 13 show the trajectories obtained from the different methods.



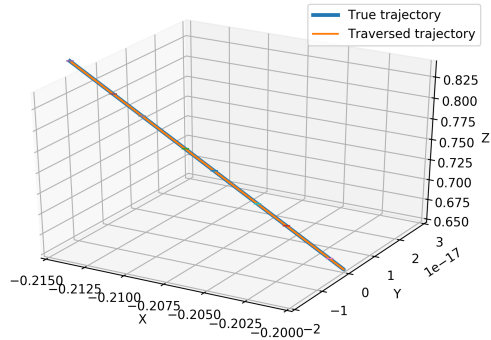
(a) Gazebo simulation preview of KUKA



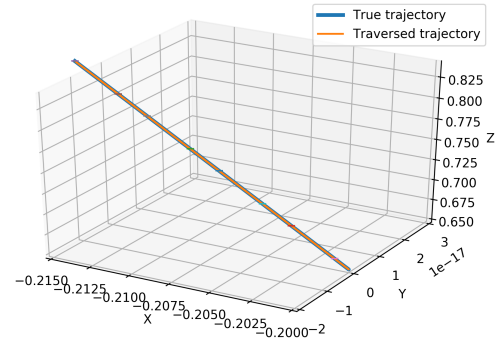
(b) Computation time for different error-tolerances



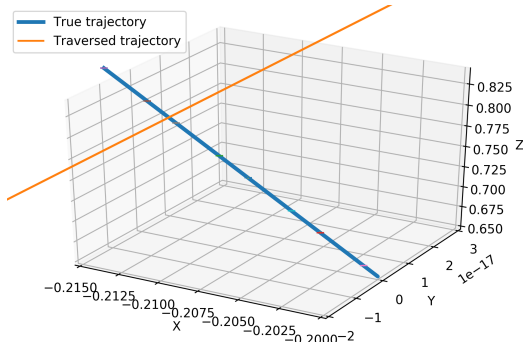
(c) Iterations taken to converge for tolerance=10⁻⁶m



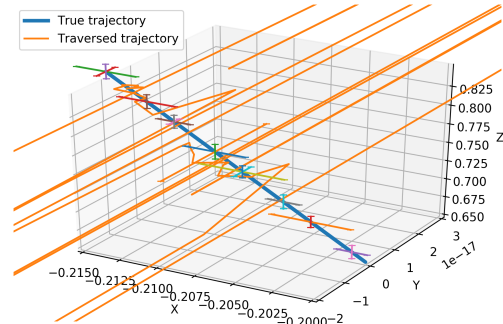
(a) Jacobian SVD



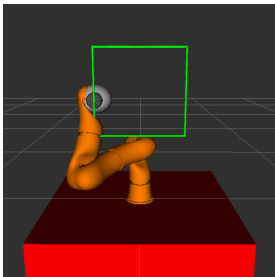
(b) Jacobian pseudo-inverse



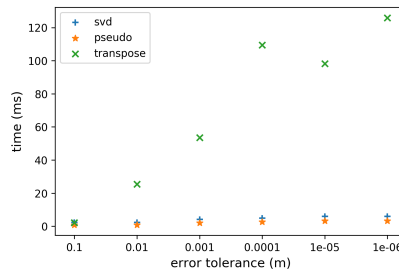
(a) Jacobian Transpose



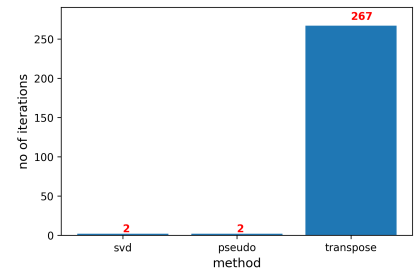
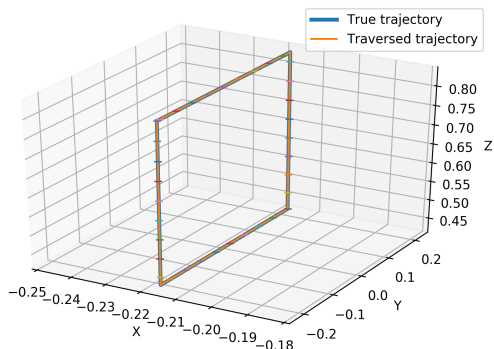
(b) Jacobian Transpose (iter = 2)

Figure 7: Linear trajectory of end-effector for different Jacobian-based algorithms with error tolerance = $10^{-6}m$.

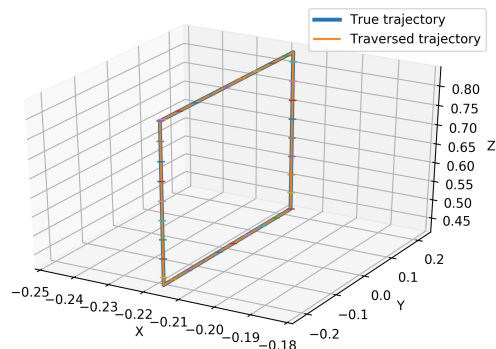
(a) Gazebo simulation preview of KUKA



(b) Computation time for different error-tolerances

(c) Iterations taken to converge for tolerance= $10^{-6}m$ 

(a) Jacobian SVD



(b) Jacobian pseudo-inverse

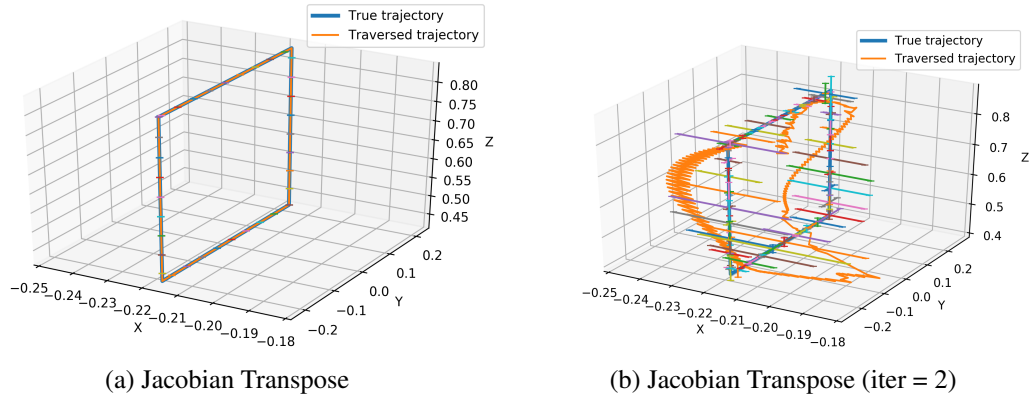
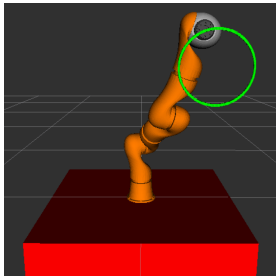
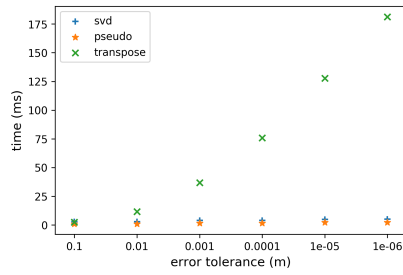


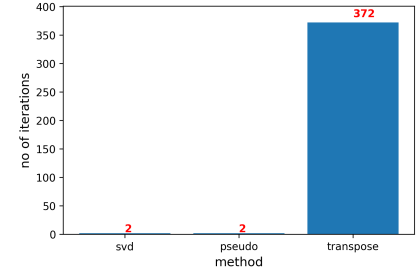
Figure 10: Square trajectory of end-effector for different Jacobian-based algorithms with error tolerance = $10^{-6}m$.



(a) Gazebo simulation preview of KUKA



(b) Computation time for different error-tolerances



(c) Iterations taken to converge for tolerance= $10^{-6}m$

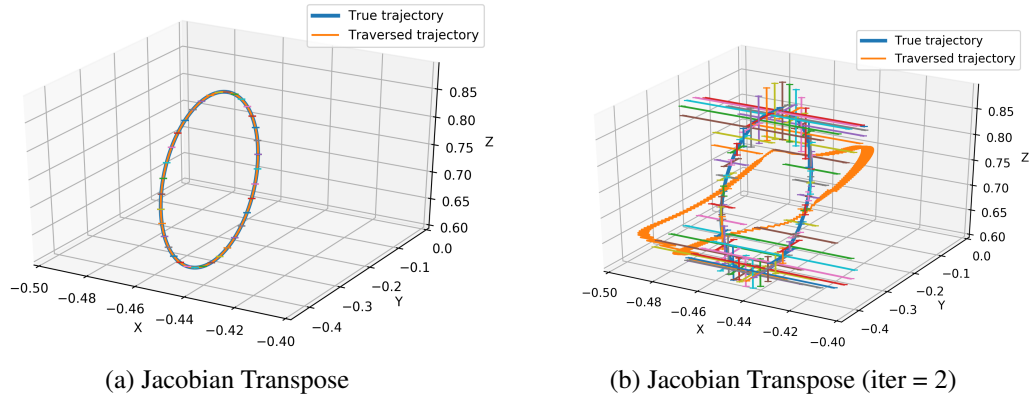
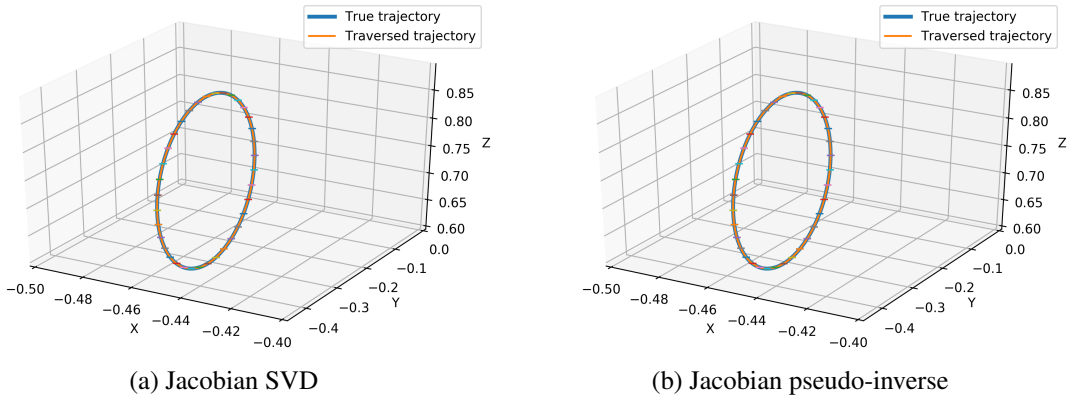


Figure 13: Circular trajectory of end-effector for different Jacobian-based algorithms with error tolerance = $10^{-6}m$.

3.2 Artificial Neural Networks

The neural network architecture specified in [6], represented in Figure 2, was implemented for validation of the study. We used a general optimiser, Adam for minimisation of the MSE loss function. The results obtained for the network, both for 3DOF and 7DOF, are shown in Figure 14. The average wall time for the neural network to output the entire circular trajectory (the highest resolution out of the three trajectories) was $\approx 15ms$. However, the initial loading of all the modules relating to the neural network was close to 20s.

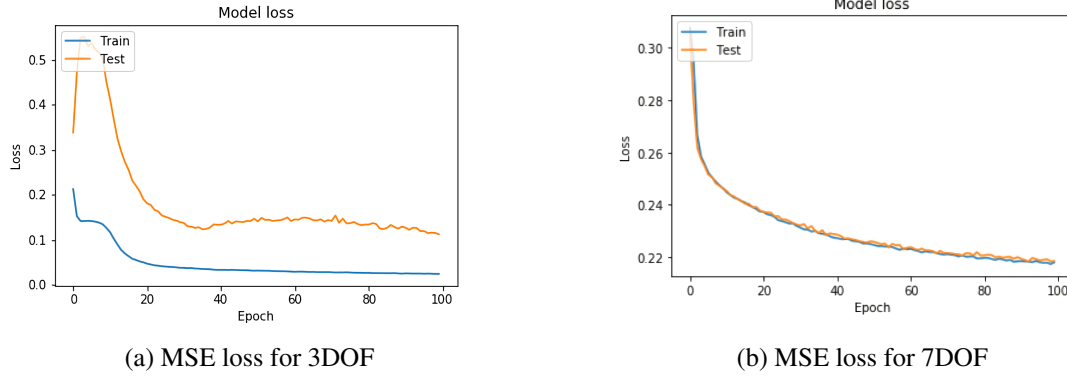


Figure 14: MSE loss for one layer perceptron with 100 hidden units.

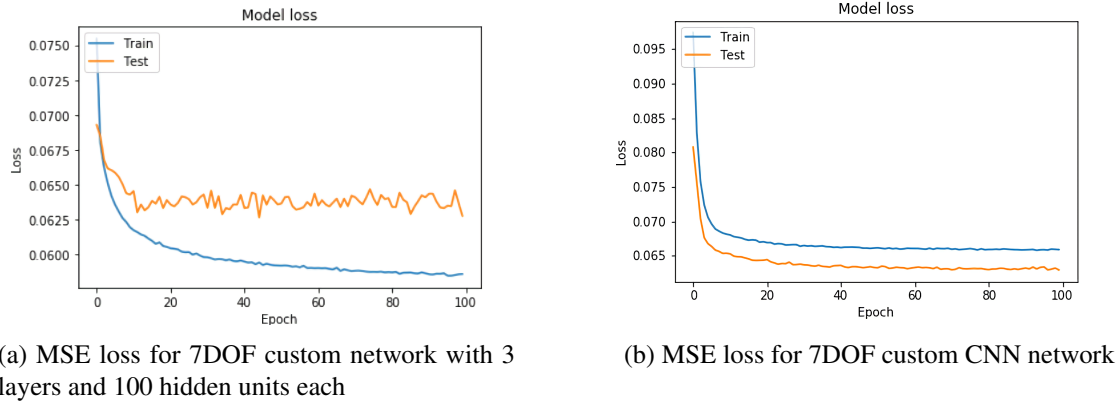


Figure 15: MSE loss for custom network architectures.

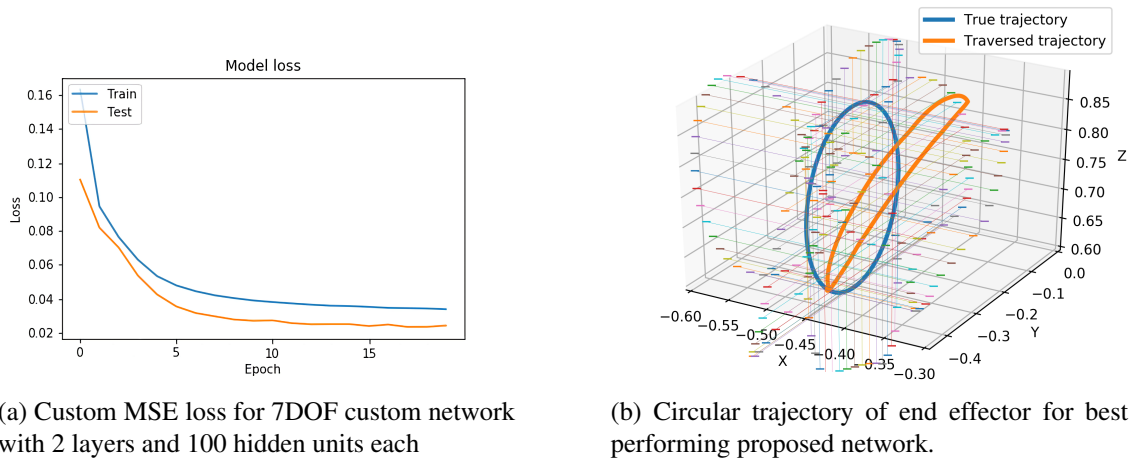


Figure 16: Custom MSE loss and network's performance for circular trajectory.

4 Discussion and Conclusions

The plots in figures 7, 10, 13 show that the Jacobian transpose method performs the worst out of all the iterative Jacobian-based methods when constrained to perform in a reactive scenario. This is due to the method's inherent slow convergence. The plots in figures 7(b), 10(b), 13(b) show the average time taken to compute each joint angles for respective trajectory for various orders of error tolerance. The Jacobian inverse using SVD and pseudo inverse perform identically well for all tolerances while Jacobian transpose got slower as required precision increased.

The following conclusions can be drawn from the results:

1. Jacobian SVD and Pseudo inverse methods were the most reactive out of the three iterative methods without compromising accuracy
2. The network described in [3] was not reproduced precisely. because of the use of a general purpose optimiser function which led to slower convergence of the loss.
3. The dense neural network has better performance on the training dataset but generalises poorly on the test dataset compared to the CNN-based architecture because of overfitting.
4. The initial loading time for the modules for neural networks is high but sequential joint angles for a trajectory are obtained faster than iterative methods because of GPU parallelisation.

5 Future work

1. Improve the training data by including the current joint angles to preserve contextual information. Also, procedurally generate data using the desired pose.
2. The entire trajectory can be fed as a time-series sequence to a recurrent neural network (RNN) or long short-term memory (LSTM) based network to save sequence context.
3. Voxel-ise the 3D work-space to get occupancy voxels to generate collision free trajectories.

References

- [1] ABB. IRB 910SC. (2019, Dec 9). [Online]. Available: <https://new.abb.com/products/robotics/industrial-robots/irb-910sc>
- [2] KUKA AG 2019. LBR iiwa — KUKA AG. (2019, Oct 11). [Online]. Available: <https://www.kuka.com/en-se/products/robotics-systems/industrial-robots/lbr-iiwa>
- [3] A. Aristidou and J. Lasenby, "FABRIK: A Fast, Iterative Solver for the Inverse Kinematics Problem," *Graph. Models*, vol. 73, no. 5, pp. 243–260, Sep. 2011. doi: 10.1016/j.gmod.2011.05.003. [Online]. Available: <http://dx.doi.org/10.1016/j.gmod.2011.05.003>
- [4] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, vol. 3, Sep. 2004. doi: 10.1109/IROS.2004.1389727 pp. 2149–2154 vol.3.
- [5] S. Bruno, S. Lorenzo, V. Luigi, and G. Oriolo, *Differential Kinematics and Statics*. London: Springer London, 2009, pp. 132–147. ISBN 978-1-84628-642-1. [Online]. Available: https://doi.org/10.1007/978-1-84628-642-1_3

- [6] Duka and Adrian-Vasile, “Neural Network based Inverse Kinematics Solution for Trajectory Tracking of a Robotic Arm,” *Procedia Technology*, vol. 12, p. 20–27, 12 2014. doi: 10.1016/j.protcy.2013.12.451
- [7] A. R. J. Almusawi, L. C. Dülger, and S. Kapucu, “A New Artificial Neural Network Approach in Solving Inverse Kinematics of Robotic Arm (Denso VP6242) Computational Intelligence and Neuroscience,” *Computational Intelligence and Neuroscience*, vol. 2016, p. 10, 2016. doi: 10.1155/2016/5720163. [Online]. Available: <https://doi.org/10.1155/2016/5720163>
- [8] A. Csiszar, J. Eilers, and A. Verl, “On solving the inverse kinematics problem using neural networks,” in *2017 24th International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, Nov 2017. doi: 10.1109/M2VIP.2017.8211457 pp. 1–6.
- [9] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: an open-source Robot Operating System,” in *ICRA Workshop on Open Source Software*, 2009.
- [10] Python. Python Software Foundation. (2020, January 10). [Online]. Available: <https://www.python.org/>
- [11] T. Oliphant. NumPy: A guide to NumPy. (2020, January 10). [Online]. Available: <http://www.numpy.org/>
- [12] NVIDIA Drivers. NVIDIA Inc. (2020, January 10). [Online]. Available: <https://www.geforce.com/drivers>
- [13] CUDA. NVIDIA Inc. (2020, January 10). [Online]. Available: <https://docs.nvidia.com/cuda/>
- [14] Docker. Docker Inc. (2020, January 10). [Online]. Available: <https://www.docker.com/>
- [15] TensorFlow. Google Inc. (2020, January 10). [Online]. Available: <https://www.tensorflow.org/>
- [16] TensorFlow Graphics. Google Inc. (2020, January 10). [Online]. Available: <https://www.tensorflow.org/graphics/overview>